

SISTEMA DE DETECCIÓN Y LECTURA DE PLACAS VEHICULARES

YOLOv8n · Streamlit · ByteTrack · SAHI grid · FastPlateOCR/ONNX · Hugging Face Spaces

Proyecto principal: detección de placas

OCR como módulo complementario

Procesamiento Digital de Imágenes



Estructura del proyecto

01 Problema y tarea

Detección de placas en imágenes y vídeos.

02 Arquitectura

YOLOv8n + filtros + OCR + tracking.

03 Entrenamiento

Configuración real, losses, mAP y curvas.

04 Funcionamiento

Ejemplos de validación, outputs y despliegue.

05 Justificación

Justificación técnica y prueba en vivo.

06 Cierre

Limitaciones y próximos pasos.

CONTEXTO DEL PROBLEMA

Automatizar localización de placas en escenarios reales

Problema real

La revisión manual de placas en imágenes o video es lenta, sensible al cansancio y difícil de escalar cuando hay múltiples vehículos, baja resolución o movimiento.

Importancia

Un detector automático puede apoyar control de acceso, revisión de registros, análisis de video y trazabilidad vehicular con resultados reproducibles.

Tarea de visión

La tarea principal es detección de objetos: localizar la placa con una caja delimitadora. La lectura del texto se realiza después sobre el crop.

POR QUÉ ES DETECCIÓN DE OBJETOS

La salida necesaria no es solo una etiqueta

Clasificación

Respondería si una imagen contiene placa, pero no entrega coordenadas.

Detección YOLO

Entrega caja delimitadora, clase y confianza; esto genera el crop para el OCR.

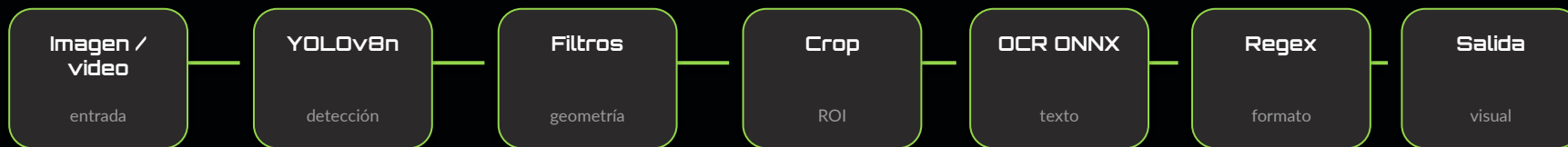
OCR

Predice caracteres dentro del crop, pero depende de que la placa haya sido localizada correctamente.

La arquitectura del sistema se diseñó como un flujo secuencial de procesamiento. En primer lugar, el modelo YOLOv8n localiza la región de interés dentro de la imagen, es decir, identifica dónde se encuentra la placa vehicular mediante una caja delimitadora. Después, esa región detectada se recorta y se normaliza para llevarla a las dimensiones y condiciones requeridas por el módulo OCR. Finalmente, el modelo OCR analiza únicamente ese recorte de la placa y predice el texto correspondiente, permitiendo obtener una salida más limpia, eficiente y coherente.

ARQUITECTURA GENERAL

Pipeline modular del sistema



Rama de video

YOLO + ByteTrack asigna ID por placa y conserva la mejor evidencia de detección/OCR por trayectoria.

Rama para placas pequeñas

La inferencia tipo SAHI por grid divide la imagen y reduce pérdida de placas pequeñas o lejanas.

ESTRUCTURA DEL REPOSITORIO

El foco operativo está en plate_dashboard/

```
.
├── OCR_Project/           # referencia y trazabilidad OCR
├── plate_dashboard/       # aplicación principal
│   ├── app.py             # interfaz Streamlit
│   ├── config.py          # rutas, umbrales, filtros
│   ├── config_tracker.yaml # ByteTrack
│   ├── Dockerfile         # despliegue en HF Space
│   ├── requirements.txt   # dependencias
│   └── models/
│       ├── best.pt        # detector YOLOv8n
│       └── ocr_best.onnx  # OCR ONNX
│   ├── src/               # módulos de procesamiento
│   └── training_docs/     # curvas, CSV, matrices, pesos
```

Separación correcta

OCR_Project no debe dominar la sustentación. La aplicación funcional que se despliega y evalúa es plate_dashboard.

Evidencia de entrenamiento

training_docs contiene args.yaml, results.csv, curvas, matrices, batches y pesos best/last.

MODELO PRINCIPAL: YOLOV8N

Variante nano para detección ligera y despliegue

YOLOv8n

modelo base: yolov8n.pt ·
checkpoint final: best.pt



Una etapa

Predice caja, clase y confianza en una sola pasada.

Nano

Modelo más liviano: favorable para Space, CPU y video.

Placas pequeñas

Localiza la región antes de recortar y aplicar OCR.

Ecosistema

Ultralytics integra train, val, plots, track y export.

CONFIGURACIÓN REAL DE ENTRENAMIENTO

Extraída de training_docs/args.yaml

Parámetros clave

task / mode	detect / train	detección
model	yolov8n.pt	YOLOv8n
epochs	300	entrenamiento largo
imgsz	640	640 px
batch	-1	auto
optimizer	auto	selección auto
seed	42	reproducible
patience	50	early stopping
cos_lr	true	scheduler
amp	true	mixed precision
close_mosaic	10	últimas 10 épocas

Lectura técnica

El entrenamiento parte de pesos preentrenados de YOLOv8n, usa 300 épocas, semilla fija, scheduler coseno, AMP y cierre de mosaic al final para estabilizar el ajuste fino.

Aumentación base

HSV, traslación, escala, flip horizontal, mosaic y erasing ayudan a simular variabilidad visual sin cambiar la etiqueta de placa.

CONFIGURACIÓN DE INFERENCIA Y FILTROS

Parámetros usados en la aplicación

```
CONF_THRESHOLD = 0.35
IMG_SIZE = 960
USE_SAH_I_IMAGE = True
SAHI_CONF_THRESHOLD = 0.20
SAHI_GRID_ROWS = 1
SAHI_GRID_COLS = 4
SAHI_GRID_OVERLAP_RATIO = 0.10
TRACK_CONF_THRESHOLD = 0.15
OCR_INPUT_HEIGHT = 64
OCR_INPUT_WIDTH = 160
OCR_CROP_PADDING = 0.08
USE_OCR_VALIDATION_FILTER = True
```

Por qué 960 en la app

Aunque el entrenamiento usa 640, la inferencia en 960 puede favorecer placas pequeñas porque aumenta resolución efectiva del objeto.

Por qué SAHI grid

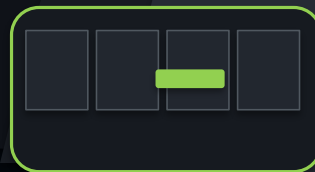
El grid 1x4 corta escenas anchas en regiones, ejecuta YOLO por sección y recompone coordenadas; mejora sensibilidad en objetos pequeños.

Por qué filtros

Aspect ratio, área, tamaño mínimo y regex reducen falsos positivos que no se comportan como placas colombianas.

SAHI para placas pequeñas

YOLOv8n sigue siendo el detector principal; SAHI mejora la visibilidad de objetos pequeños.



Más escala relativa por recorte

QUÉ ES

SAHI divide la imagen en recortes.

YOLOv8n se ejecuta en cada sección para ver mejor objetos pequeños.

POR QUÉ

Las placas pueden ser lejanas.

El grid aumenta detalle efectivo sin cambiar el modelo ni reentrenar.

CÓMO SE USÓ

Implementación propia tipo SAHI.

Grid 1×4, solape 10%, reproyección de cajas y NMS.

Flujo aplicado

Imagen

Grid 1×4

YOLOv8n

Reproyección

NMS final

Placas

Configuración: USE_SAH_IMAGE=True · USE_SAH_VIDEO=False · GRID_ROWS=1 · GRID_COLS=4 · OVERLAP=0.10 · NMS_IΟΥ=0.45

Idea clave: SAHI no reemplaza a YOLOv8n; lo ayuda a recuperar placas pequeñas en imágenes.

FUNCIÓN DE PÉRDIDA Y MÉTRICAS

Cómo interpretar el entrenamiento

Box loss

Mide error de localización. Si la caja no cubre bien la placa real, sube la pérdida.

Class loss

Mide error de clasificación. En este proyecto la clase principal es placa, por eso la dificultad dominante es localizar.

OFL loss

Refina la regresión de los bordes; es importante para crops pequeños donde pocos píxeles alteran el OCR.

Precision

De todas las detecciones, qué porcentaje eran realmente placas. Controla falsos positivos.

Recall

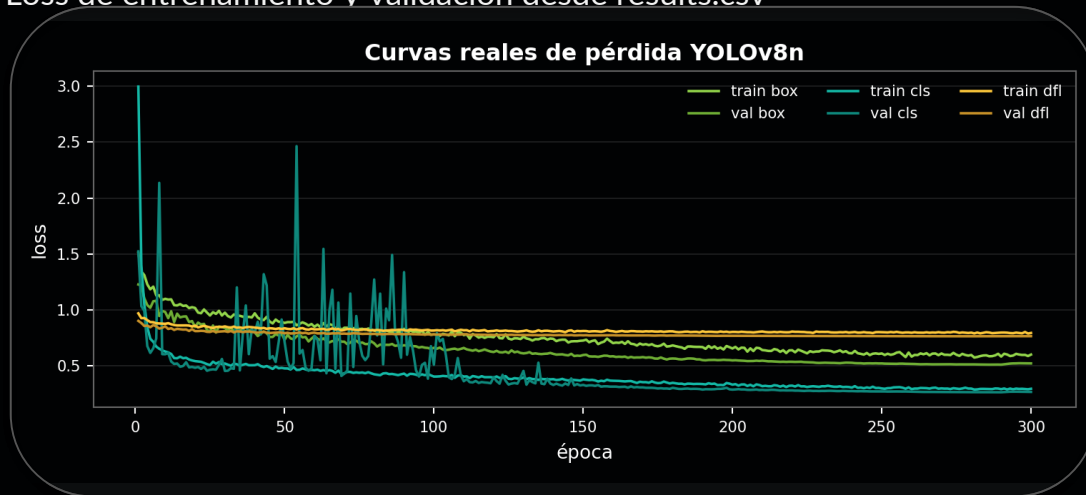
De todas las placas reales, qué porcentaje encontró el detector. Controla falsos negativos.

mAP50-95

Métrica estricta: evalúa precisión promedio con múltiples umbrales de IoU, no solo IoU=0.50.

CURVAS REALES DE PÉRDIDA

Loss de entrenamiento y validación desde results.csv



Lectura de la curva

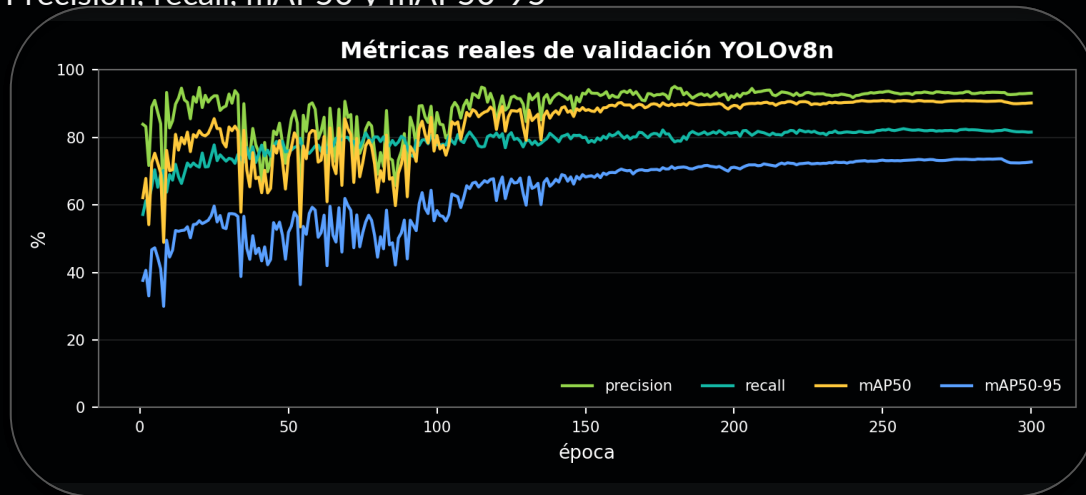
Las pérdidas de entrenamiento y validación descenden y se estabilizan. El patrón sugiere aprendizaje efectivo sin una divergencia fuerte entre train y val.

Valores finales

epoch 300: train box 0.602, cls 0.297, dfl 0.795; val box 0.524, cls 0.269, dfl 0.766.

CURVAS REALES DE VALIDACIÓN

Precision, recall, mAP50 v mAP50-95



93.1%

precision final

81.6%

recall final

90.2%

mAP50 final

72.7%

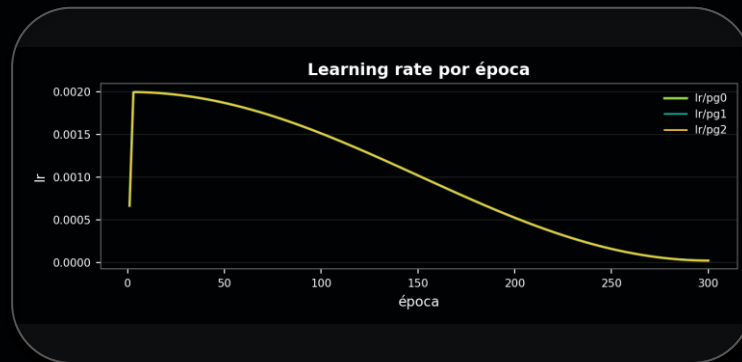
mAP50-95 final

RESUMEN CUANTITATIVO YOLOV8N

Mejores épocas y desempeño final

Métricas principales

Final epoch	300	mAP50 90.20%
Best mAP50	epoch 250	90.93%
Best mAP50-95	epoch 290	73.66%
Best F1	epoch 252	87.68%
Final precision	epoch 300	93.09%
Final recall	epoch 300	81.58%

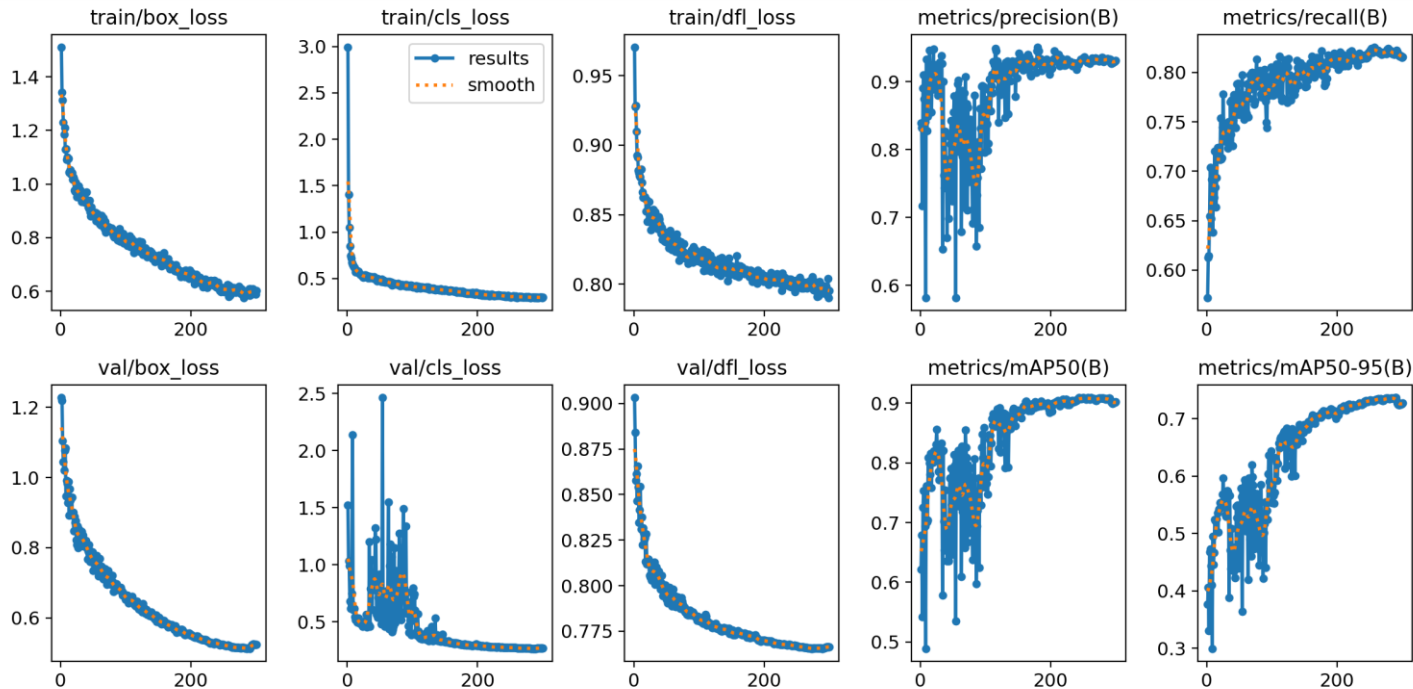


Interpretación

El mejor mAP50-95 aparece cerca de la época 290. La curva final mantiene valores altos, por lo que best.pt debe corresponder al mejor checkpoint guardado por validación.

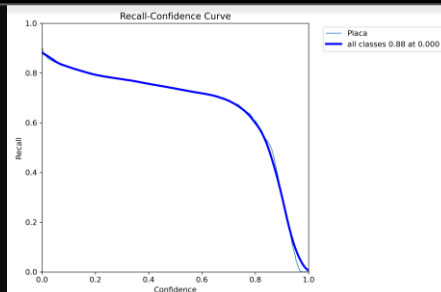
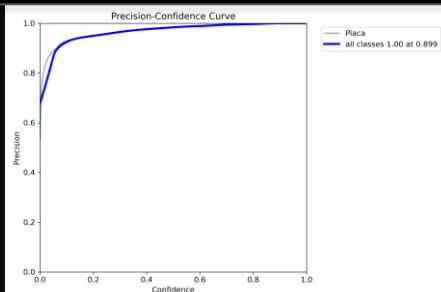
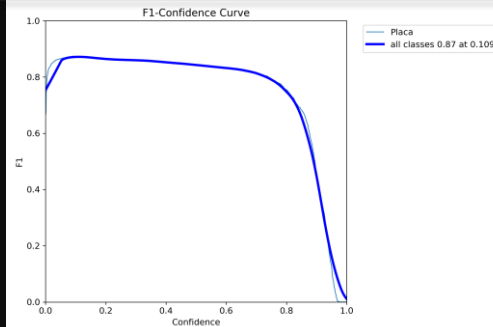
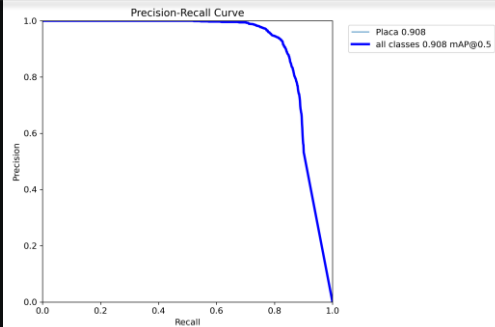
RESUMEN GRÁFICO DE ULTRALYTICS

results.png generado por el entrenamiento



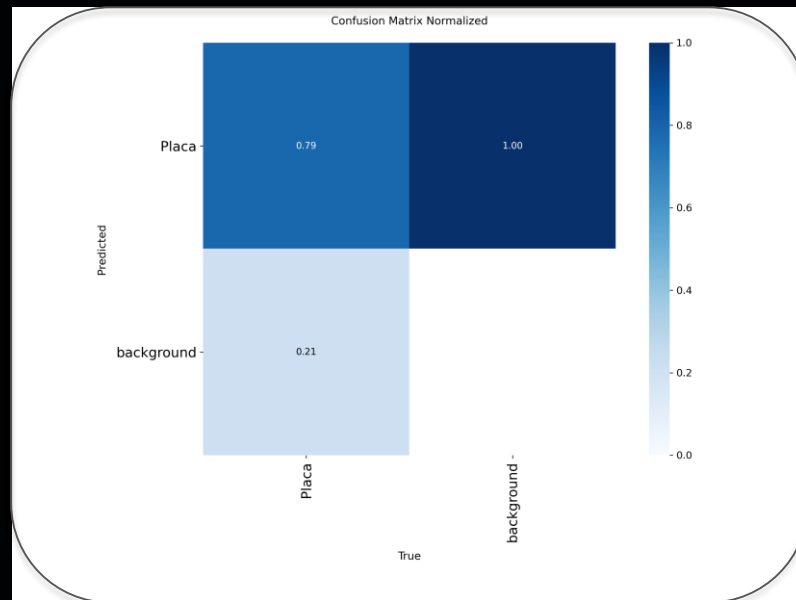
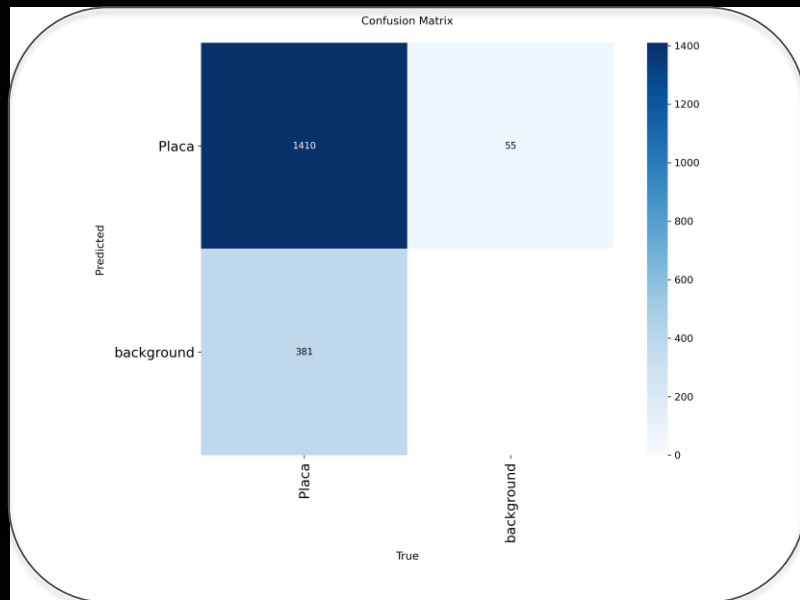
CURVAS DE DIAGNÓSTICO DEL DETECTOR

Precision/Recall, F1, P y R vs confianza



MATRIZ DE CONFUSIÓN

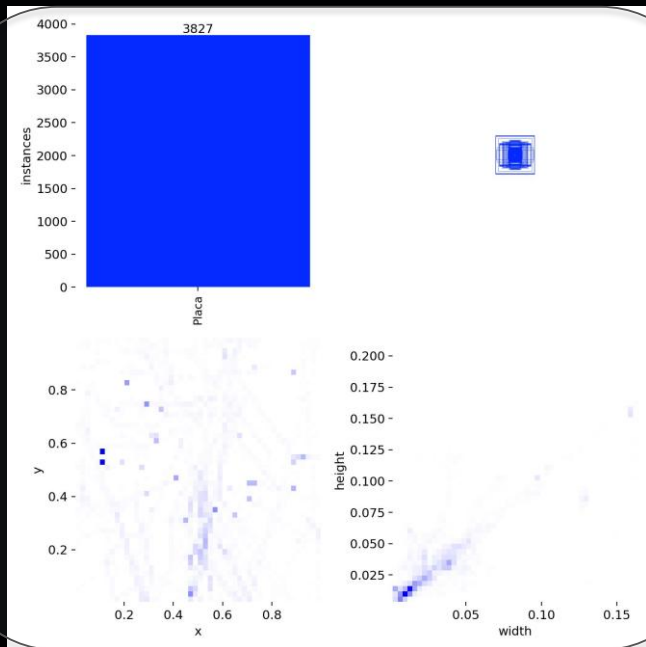
Errores de clase y fondo



La matriz ayuda a distinguir dos tipos de fallo: placas no detectadas y detecciones confundidas con fondo.

EVIDENCIA VISUAL DEL DATASET

Distribución de etiquetas y batch de entrenamiento



EJEMPLOS DE VALIDACIÓN

Comparación etiquetas vs predicciones



val_batch0_labels.jpg



val_batch0_pred.jpg

Cómo leerlo

A la izquierda están las etiquetas; a la derecha las predicciones del modelo. Esto permite revisar visualmente si la caja cae sobre la placa y si aparecen falsos positivos.

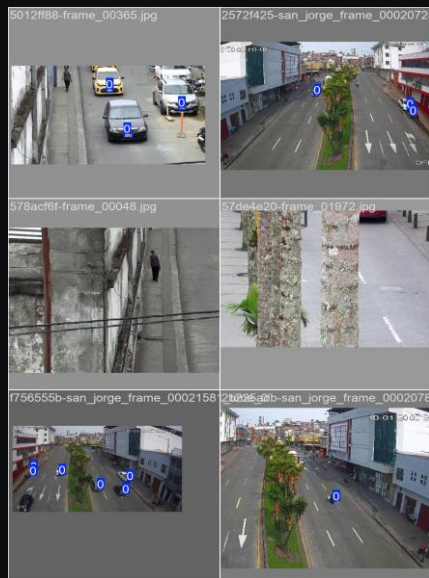
05 RESULTADOS ADICIONALES DE VALIDACIÓN

Variabilidad de escenas y cajas detectadas



BATCHES GENERADOS AL FINAL DEL ENTRENAMIENTO

Revisión cualitativa de augmentación y escala



FUNCIONAMIENTO EN VIDEO

YOLOv8n + ByteTrack + memoria por ID

Frame t

detecta placa

Frame t+1

conserva ID

Track summary

mejor evidencia

Memoria por track_id

Para cada placa se conserva best_det_conf, best_ocr_text, best_ocr_conf y class_id. Así, un frame malo no borra la mejor lectura previa.

Métrica recomendada en video

Además de frame_plate_acc, conviene reportar track_plate_acc: exactitud final por trayectoria de vehículo.

MÓDULO OCR COMPLEMENTARIO

FastPlateOCR exportado a ONNX

Rol del OCR

No localiza placas: recibe el crop generado por YOLOv8n y predice la secuencia de caracteres.

Modelo usado

ocr_best.onnx para inferencia;
ocr_best.keras para continuar fine-tuning.

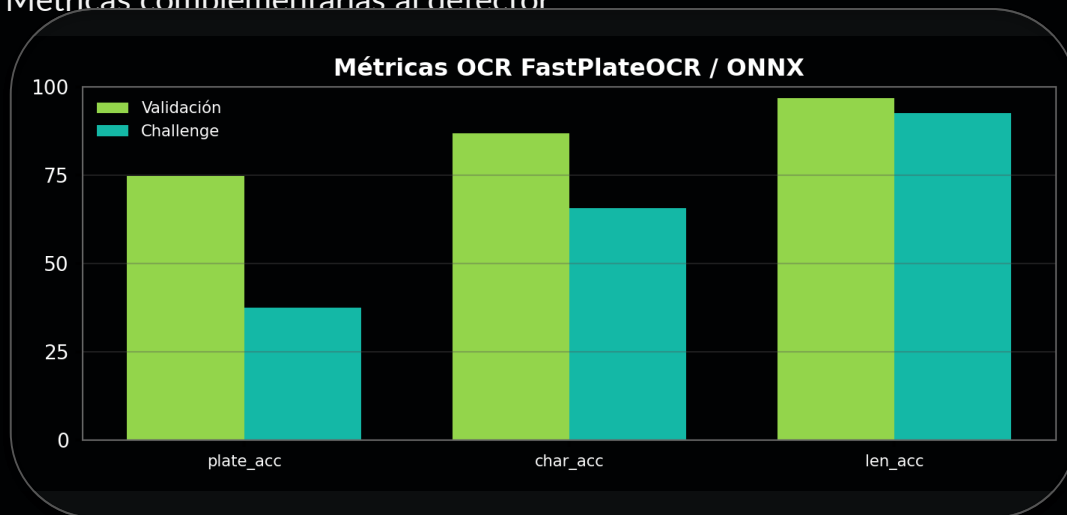
Entrada

RGB 160×64, longitud máxima 9, alfabeto 0-9 + A-Z y padding '_ '.

```
video / imagen  
→ YOLOv8n detector  
→ crop con margen  
→ normalización 160×64 RGB  
→ OCR FastPlateOCR / ONNX  
→ regex colombiano  
→ voto temporal por track
```

RESULTADOS DEL OCR

Métricas complementarias al detector



74.82%

plate_acc val

86.92%

char_acc val

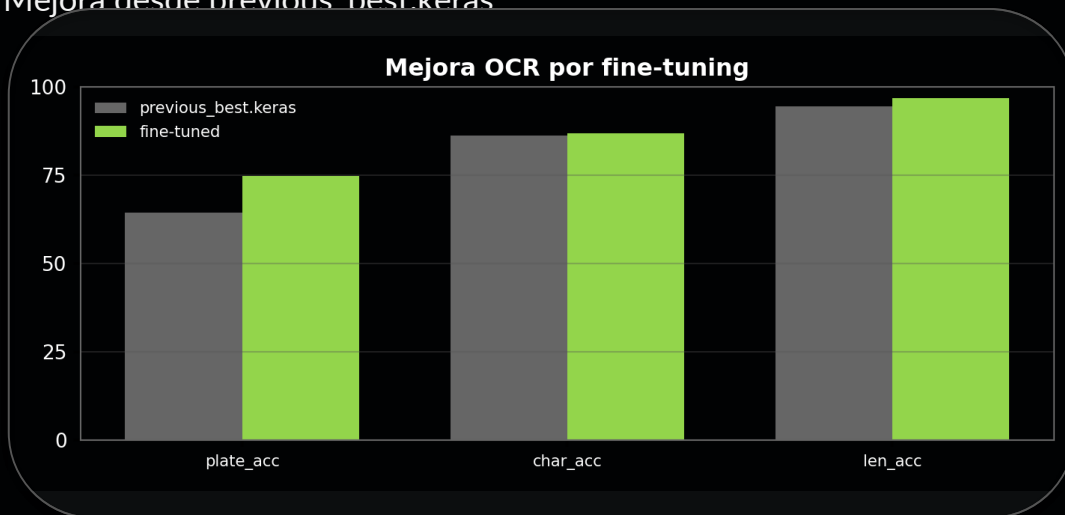
96.98%

len_acc val

El challenge es más extremo que el caso promedio; en video, la fusión temporal puede mejorar la decisión final.

FINE-TUNING OCR

Mejora desde previous_best.keras



Estrategia

Se continuó desde previous_best.keras porque el modelo anterior ya capturaba mejor el dominio de placas que el modelo global inicial.

Mejora principal

Exact plate accuracy subió de 64.35% a 74.82%, una ganancia de +10.47 puntos.

POSTPROCESO COLOMBIANO

Validación de formato y corrección controlada

Formatos considerados:

ABC123	carro
ABC12D	moto actual
ABC12	moto antigua
123ABC	mototaxi
R12345 / S12345	remolque
T1234	carrotanque
CC1234	consular
OI1234	organización internacional
AT1234	administrativo técnico
123456	policía simplificado

Confusiones típicas

O↔0, I↔1, Z↔2, S↔5, G↔6, B↔8, T↔7. Solo se corrigen si la posición en el formato lo justifica.

Votación temporal

Si la misma placa aparece en varios frames, se decide por consenso y confianza acumulada, no por un único frame aislado.

DESPLIEGUE EN HUGGING FACE SPACES

Interfaz interactiva para la prueba en caliente

Carpeta base

El Space debe construirse desde plate_dashboard/, donde están app.py, config.py, Dockerfile, requirements.txt, models/ y src/.

Runtime

Streamlit expone la app en puerto 7860; Docker instala ffmpeg, libgl, dependencias de Python y OpenCV headless.

```
CMD ["streamlit", "run", "app.py",  
    "--server.address=0.0.0.0",  
    "--server.port=7860"]
```

Archivos críticos

models/best.pt y models/ocr_best.onnx deben estar accesibles. Si pesan mucho, usar Git LFS.

PRUEBA EN CALIENTE

Coherencia entre dataset, modelo y Space

1

Dataset reportado

El README debe decir exactamente qué base de datos se usó.

2

Imagen aleatoria

El evaluador selecciona una muestra cualquiera.

3

Inferencia Space

Se sube al dashboard en Hugging Face.

4

Resultado

Caja de placa y OCR coherente con la imagen.

Qué se debe defender

El sistema debe funcionar con una imagen no seleccionada manualmente. Por eso las curvas, el dataset y el Space deben estar alineados.

EXPORTACIÓN Y TRAZABILIDAD

Requisito adicional: documentación con ExecuTorch

Detector YOLOv8n

Entrenado desde yolov8n.pt y conservado como best.pt / last.pt.
Documentar comando, dataset, métricas y checkpoint.

OCR ONNX

El OCR ya está en formato ONNX para inferencia rápida. Debe quedar claro que es módulo complementario.

Documentar en training_docs/:

1. dataset y particiones
2. comando/script de entrenamiento
3. results.csv y results.png
4. best.pt generado
5. proceso de exportación con ExecuTorch
6. validación del artefacto desplegado

Punto crítico

Aunque la aplicación use best.pt/ONNX, los lineamientos piden dejar trazabilidad del proceso de exportación con ExecuTorch para integración posterior.

CONCLUSIÓN

Sistema modular, medible y desplegable

La solución se defiende como un proyecto de detección de objetos aplicado a placas vehiculares. YOLOv8n es el núcleo porque localiza la placa de forma eficiente; el OCR ONNX, los filtros, SAHI grid y ByteTrack completan el flujo para imágenes y videos.

Resultado del detector

mAP50 final 90.21%, mAP50-95 final 72.72%, precision final 93.09%, recall final 81.58%.

Mensaje de defensa

Detectar bien la placa es la condición necesaria para que OCR, tracking y validación produzcan una lectura confiable.



¡GRACIAS!